总结：Agent 层干什么？

Agent 层 = 智能体的核心大脑 + 行动能力

它只负责一件事：
定义「这个助手如何思考、如何决策、如何调用工具、如何形成完整工作流」。

具体包括：

- 定义 State
- 编写节点函数（LLM 节点、工具节点等）
- 定义路由逻辑（条件边 / Command）
- 构建并编译图
- 定义可用工具
- 实现记忆、人工介入、多轮对话等控制逻辑

每次写代码时，如何快速判断写在哪一层？

用这个简单判断法：

你要写的内容是什么？	应该放在哪一层	原因
图怎么建、节点怎么写、路由怎么跳、工具怎么定义	**agents/**	智能体核心逻辑
对外提供接口（HTTP、WebSocket 等）	**api/**	对外入口
数据结构定义（State、请求/响应模型）	**models/**	数据格式
真实或模拟数据、数据库操作	**db/**	数据访问
配置、日志、通用工具函数	**common/**	基础设施
提示词文件、静态资源	**resources/** 或 **static/**	资源文件

快速口诀：

- 跟「智能体如何思考和行动」有关 → **agents**
- 跟「怎么被外部调用」有关 → **api**
- 跟「数据长什么样」有关 → **models**
- 跟「数据从哪来」有关 → **db**
- 跟「配置和公共支持」有关 → **common**

Agent 层会复杂控制所有层吗?

不会。

Agent 层**不控制**其他层，而是**被其他层服务**。

关系是这样的：

```
text

api 层（入口）
    ↓ 调用
agents 层（核心大脑）
    ↓ 使用
models + db + common + tools
```

- **api** 调用 agents
- **agents** 使用 models、db、common、tools
- agents **不主动控制** api、db 等层

这是标准的分层架构：**上层依赖下层，下层不依赖上层**。

最终一句话总结：

Agent 层只专注于「智能体本身怎么工作」，其它层是为它服务的。
写代码时先问自己：“这是智能体思考/行动的逻辑吗？”——是就放 agents，不是就放对应层。